



# Politechnika Wrocławska

|                                                                 |                                                                                     |                                             |
|-----------------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------|
| Sterowanie procesami dyskretnymi<br>Automatyka i Robotyka, W12N | Zadanie nr 1<br>Minimalizacja maksymalnej nieterminowości<br>Problem jednomaszynowy |                                             |
| Jakub Borsukiewicz 278319<br>Jan Farbotko 278325                | Termin zajęć:<br>Wt. Tn. 11:15                                                      | Prowadzący:<br>dr inż.<br>Agnieszka Wielgus |

# Spis treści

|          |                                                                                       |           |
|----------|---------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Opis problemu matematycznego</b>                                                   | <b>3</b>  |
| <b>2</b> | <b>Zaimplementowane algorytmy</b>                                                     | <b>3</b>  |
| 2.1      | Algorytm heurystyczny oparty o sortowanie po $r_j$ (ERD) . . .                        | 3         |
| 2.2      | Algorytm heurystyczny oparty o sortowanie po $d_j$ (EDD) . . .                        | 3         |
| 2.3      | Przegląd zupełny . . . . .                                                            | 4         |
| 2.4      | Algorytm konstrukcyjny własnego autorstwa . . . . .                                   | 4         |
| 2.4.1    | Opis algorytmu . . . . .                                                              | 4         |
| 2.4.2    | Pseudokod . . . . .                                                                   | 4         |
| 2.5      | Algorytm Schrage . . . . .                                                            | 5         |
| 2.6      | Algorytm Schrage z zadaniami podzielными . . . . .                                    | 5         |
| 2.7      | Schemat B&B (algorytm Carliera) . . . . .                                             | 5         |
| <b>3</b> | <b>Testy eksperymentalne</b>                                                          | <b>6</b>  |
| 3.1      | Analiza porównawcza wydajności czasowej algorytmów . . . .                            | 8         |
| 3.2      | Analiza porównawcza jakości algorytmów w oparciu o kryte-<br>rium $L_{max}$ . . . . . | 8         |
| 3.3      | Analiza jakości algorytmu własnego . . . . .                                          | 9         |
| <b>4</b> | <b>Wnioski</b>                                                                        | <b>10</b> |

# 1 Opis problemu matematycznego

Dana jest jedna maszyna, która ma do wykonania pewien zbiór zadań  $n$   $J = 1, \dots, n$ . Zadanie  $j$  charakteryzuje się następującymi parametrami:

- $n_j$  - termin dostępności
- $d_j$  - pożądaný termin zakończenia wykonania zadania  $j$
- $p_j$  - czas wykonania zadania

Niech przez  $C_j$  oznaczmy zakończenie wykonania zadania  $j$ . Problemem jest minimalizacja maksymalnej nieterminowości, którą obliczamy ze wzoru  $L_j = C_j - d_j$ .

## 2 Zaimplementowane algorytmy

Zaimplementowano różne algorytmy, których celem było rozwiązanie otrzymanego problemu.

### 2.1 Algorytm heurystyczny oparty o sortowanie po $r_j$ (ERD)

- Zasada działania - sortowanie rosnąco po terminach dostępności  $r_j$
- Cechy szczególne - dąży do maksymalizacji wykorzystania maszyny od samego początku
- Złożoność -  $O(n \log(n))$

### 2.2 Algorytm heurystyczny oparty o sortowanie po $d_j$ (EDD)

- Zasada działania - sortowanie rosnąco po terminach zakończenia  $d_j$
- Cechy szczególne - skupia się na pilności zadań, ale może generować duże przestoje jeśli zadanie z małym  $d_j$  ma duże  $r_j$
- Złożoność -  $O(n \log(n))$

## 2.3 Przegląd zupełny

- Zasada działania - algorytm generuje wszystkie możliwe permutacje i z nich oblicza kryterium  $L_{max}$
- Cechy szczególne - algorytm gwarantuje znalezienie rozwiązania optymalnego, ale jest niepraktyczny dla dużych danych
- Złożoność -  $O(n!)$

## 2.4 Algorytm konstrukcyjny własnego autorstwa

### 2.4.1 Opis algorytmu

Algorytm ten polega na przypisaniu dostępnym aktualnie zadaniom priorytetów na podstawie wzoru:  $priorytet = \alpha \cdot d_j + (1 - \alpha) \cdot r_j$ . Współczynnik  $\alpha$  można dobrać z zakresu 0-1, w zależności od tego co uważamy za ważniejszy parametr. Następnie zadania będą wykonywane na podstawie tego priorytetu.

### 2.4.2 Pseudokod

#### 1. Inicjalizacja

- Utwórz pustą permutację
- Oznacz wszystkie zadania jako niewykonane
- Ustal aktualny\_czas jako 0
- Ustal współczynnik wartość  $\alpha$

#### 2. Pętla - powtarzaj n razy (wybierz kolejne zadanie):

- Szukaj dostępnych niewykonanych zadań gdzie  $r_j \leq \text{czas\_aktualny}$ 
  - Dla każdego dostępnego czasu oblicz priorytet:  
 $priorytet = \alpha \cdot d_j + (1 - \alpha) \cdot r_j$
  - Wybierz zadanie z najmniejszym priorytetem
- Jeśli brak dostępnych zadań:
  - Znajdź niewykonane zadanie z najmniejszym  $r_j$
  - Przesuń czas:  $\text{czas\_aktualny} = r_j$  tego zadania
- Wybranie najlepszego zadania:
  - Dopisz wybrane najlepsze zadanie do permutacji
  - Oznacz jako wykonane

–  $\text{czas\_aktualny} += p_j$

3. Oblicz i zwróć wartość kryterium  $L_{max}$  dla uzyskanej permutacji

## 2.5 Algorytm Schrage

- Zasada działania - algorytm zachłanny, który w każdym kroku wybiera spośród dostępnych zadań to, które ma najmniejszy termin zakończenia  $d_j$
- Cechy szczególne - algorytm jest skuteczniejszy od prostego EDD, ponieważ bierze pod uwagę zmieniający się w czasie zbiór zadań gotowych do realizacji. Może wykorzystywać kolejkę priorytetową.
- Złożoność -  $O(n^2)$  (wersja podstawowa) lub  $O(n \log(n))$  (przy użyciu kolejki priorytetowej)

## 2.6 Algorytm Schrage z zadaniami podzielnymi

- Zasada działania - rozszerzenie algorytmu Schrage pozwalające na przerywanie wykonywania zadania gdy na maszynie pojawi się inne zadanie o mniejszym  $d_j$
- Cechy szczególne - wynik tego algorytmu stanowi dolne ograniczenie (lower bound) dla wersji bez podziału i jest kluczowe w schemacie B&B
- Złożoność -  $O(n \log(n))$

## 2.7 Schemat B&B (algorytm Carliera)

- Zasada działania - algorytm wykorzystuje metodę podziału i ograniczeń (Branch and Bound). Buduje drzewo poszukiwań, eliminując gałęzie, które nie mogą dać wyniku lepszego niż aktualnie znaleziony. Korzysta z algorytmu Schrage i Schrage z podziałami
- Cechy szczególne - pozwala na znalezienie rozwiązania optymalnego dla dużych instancji (większych niż dla przeglądu zupełnego)
- Złożoność -  $O(2^n \cdot n \log(n))$

### 3 Testy eksperymentalne

Przeprowadzono testy na algorytmach w celu przeprowadzenia analizy porównawczej ich wydajności czasowej oraz jakości generowanych rozwiązań przez algorytmy w oparciu o wyznaczone wartości kryterium  $L_{max}$ .

Testy wykonano na instancjach o wielkości z zakresu 4 - 12. Instancje wygenerowano losowo. Następnie zrobiono również testy na danych z Benchmarka: 100\_Independent0290, 0297, 0360, 0376 i 0428.

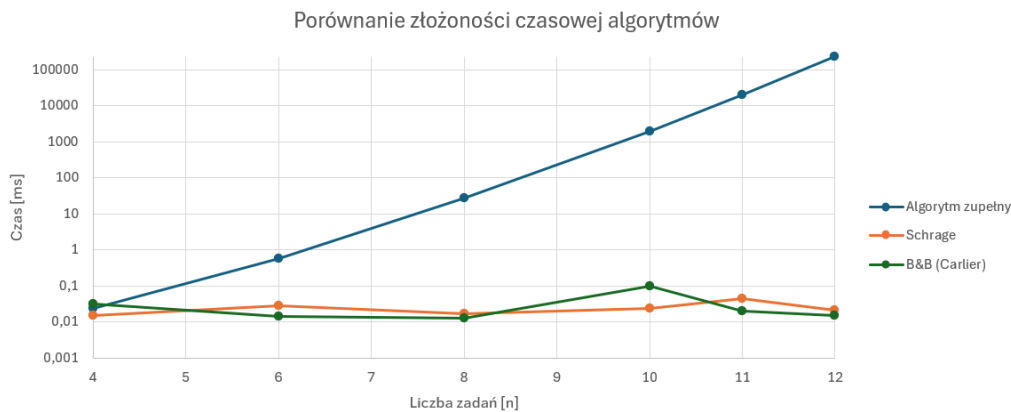
Otrzymane wyniki przedstawiono w tabeli 1.

Tabela 1: Wyniki eksperymentów

| n      | Sortuj<br>Lmax | Sortuj<br>Bład | Sortuj t<br>[ms] | Sortuj<br>dł<br>Lmax | Sortuj<br>dł<br>Bład | Sortuj<br>t [ms] | Optymalny<br>Lmax (opt) | Optymalny<br>t [ms] | Schrage<br>Lmax | Schrage<br>Bład | Schrage<br>t [ms] | Schrage<br>PMTN<br>Lmax | PMTN t<br>[ms] | B&B<br>Lmax | B&B<br>Bład | B&B t<br>[ms] | Własny<br>(0,5)<br>Lmax | Własny<br>Bład | Własny<br>[ms] |
|--------|----------------|----------------|------------------|----------------------|----------------------|------------------|-------------------------|---------------------|-----------------|-----------------|-------------------|-------------------------|----------------|-------------|-------------|---------------|-------------------------|----------------|----------------|
| 4      | -2             | 33.33%         | 0.02             | -3                   | 0.00%                | 0.01             | -3                      | 0.024               | -2              | 33.33%          | 0.015             | -3                      | 0.051          | -3          | 0.00%       | 0.032         | -2                      | 33.33%         | 0.031          |
| 6      | -2             | 0.00%          | 0.029            | 0                    | 100.00%              | 0.013            | -2                      | 0.575               | -2              | 0.00%           | 0.029             | -2                      | 0.025          | -2          | 0.00%       | 0.014         | -2                      | 0.00%          | 0.02           |
| 8      | -2             | 0.00%          | 0.029            | 0                    | 100.00%              | 0.02             | -2                      | 28.183              | -2              | 0.00%           | 0.017             | -2                      | 0.026          | -2          | 0.00%       | 0.013         | -2                      | 0.00%          | 0.026          |
| 10     | 4              | 33.33%         | 0.029            | 6                    | 100.00%              | 0.02             | 3                       | 1965.21             | 4               | 33.33%          | 0.024             | 2                       | 0.024          | 3           | 0.00%       | 0.1           | 4                       | 33.33%         | 0.023          |
| 11     | 5              | 0.00%          | 0.021            | 9                    | 80.00%               | 0.019            | 5                       | 19882               | 5               | 0.00%           | 0.044             | 5                       | 0.027          | 5           | 0.00%       | 0.02          | 5                       | 0.00%          | 0.046          |
| 12     | 8              | 0.00%          | 0.024            | 12                   | 50.00%               | 0.018            | 8                       | 239076              | 8               | 0.00%           | 0.021             | 8                       | 0.025          | 8           | 0.00%       | 0.015         | 8                       | 0.00%          | 0.029          |
| 100(1) | 4242           | 380%           | 0.100            | 1362                 | 54.07%               | 0.088            | —                       | —                   | 901             | 1.92%           | 0.199             | 884                     | 0.230          | 884         | 0.00%       | 11.64         | 901                     | 1.92%          | 1.028          |
| 100(2) | 4268           | 346%           | 0.083            | 1341                 | 40.27%               | 0.084            | —                       | —                   | 967             | 1.15%           | 0.318             | 956                     | 0.353          | 956         | 0.00%       | 19.64         | 970                     | 1.46%          | 0.792          |
| 100(3) | 3353           | 1075%          | 0.068            | -236                 | 31.40%               | 0.062            | —                       | —                   | -268            | 25.00%          | 0.238             | -344                    | 0.332          | -344        | 0.00%       | 8.12          | -291                    | 15.41%         | 0.77           |
| 100(4) | 3444           | 2560%          | 0.077            | -140                 | 0.00%                | 0.069            | —                       | —                   | -120            | 14.29%          | 0.252             | -140                    | 0.371          | -140        | 0.00%       | 11.08         | -120                    | 14.29%         | 0.801          |
| 100(5) | 1887           | 228%           | 0.086            | -1321                | 11.87%               | 0.057            | —                       | —                   | -1415           | 5.60%           | 0.217             | -1499                   | 0.253          | -1499       | 0.00%       | 20.36         | -1448                   | 3.40%          | 0.827          |

### 3.1 Analiza porównawcza wydajności czasowej algorytmów

Na podstawie wyników dla instancji o rozmiarach 4 - 12 wykreślono wykres zależności czasu potrzebnego na wykonanie algorytmu od ilości zadań w instancji. Wykres przedstawiono na rysunku 1.



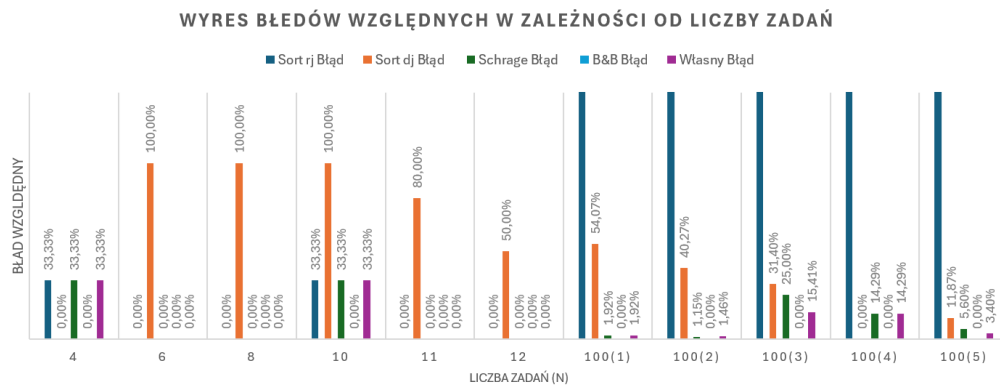
Rysunek 1: Wykres porównania złożoności czasowej algorytmów

(Oś Y wykresu została przedstawiona w skali algorytmicznej, ponieważ wyniki jednego algorytmu bardzo odbiegają od innych). Z wykresu można zaobserwować, że algorytmy Schrage i Carliera działają szybko, co odpowiada ich złożoności obliczeniowej która jest logarytmiczna. W przypadku Algorytmu zupełnego widoczny jest ogromny wzrost czasu przy kolejnych zwiększaniach ilości zadań. Widoczna jest jego złożoność  $O(n!)$ . Oznacza to, że algorytm zupełny jest dobry tylko dla instancji o małym rozmiarze.

### 3.2 Analiza porównawcza jakości algorytmów w oparciu o kryterium $L_{max}$

Na podstawie wyników tabeli wykreślono wykres słupkowy błędów względnych kryterium  $L_{max}$  od ilości zadań. Wykres przedstawiono na rysunku 2





Rysunek 2: Wykres porównania jakości algorytmów

Na pierwszych sześciu wykresach słupkowych można zauważyć, że wyniki są różne, nie ma tam większego schematu. Wynika to z małej ilości zadań do posortowania, a efektywność algorytmu zależy przede wszystkim od wartości parametrów zadań na których są wykonywane operacje sortowania.

Na wykresach słupkowych, w których rozmiar instancji wynosi 100 można zaobserwować pewne zależności. Algorytm sortowania po parametrze  $r_j$  za każdym razem osiąga ogromny błąd. Lepiej zachowuje się algorytm sortowania po parametrze  $d_j$ , jednak wciąż może to wynosić nawet kilkadziesiąt procent. (W jednym z testów, sortowanie po  $d_j$  dało wynik idealny, co oznacza, że jest też to możliwe). Najlepiej radzi sobie algorytm Schrage, które wyniki są lepsze (jednak występują też większe błędy, ze względu na zachłanność algorytmu). Wielkim zaskoczeniem jest algorytm własny, który jak się okazuje jest również dobry co Schrage, a nawet i czasami lepszy.

### 3.3 Analiza jakości algorytmu własnego

Przeprowadzono test własnego algorytmu, w którym ważnym elementem był dobór parametru priorytetu. W tym celu wykonano eksperyment poprzez wykonanie wielu operacji w pętli, w której parametr był zmieniany w zakresie 0 - 1 ze skokiem 0,02 oraz 0,001. W wyniku otrzymano takie parametry jak: najlepsza/najgorsza  $\alpha$ , błąd min/max/średni względem optymalnego oraz błąd względny i czasy wykonania algorytmu (pętli operacji). Wyniki przedstawiają tabele 2 i 3.

Tabela 2: Wyniki eksperymentu dla 50 operacji

| n      | Best $\alpha$ | min. błąd | Worst $\alpha$ | max. błąd | średni błąd | Względny błąd (B&B) | Czas [ms] |
|--------|---------------|-----------|----------------|-----------|-------------|---------------------|-----------|
| 100(1) | 0,42          | 17        | 0              | 3358      | 565,451     | 1,92%               | 29,689    |
| 100(2) | 0,98          | 11        | 0              | 3312      | 656,922     | 1,15%               | 29,992    |
| 100(3) | 0,74          | 48        | 0              | 3697      | 824         | 13,95%              | 32,25     |
| 100(4) | 0,42          | 20        | 0              | 3584      | 476,255     | 14,23%              | 29,455    |
| 100(5) | 0,52          | 6         | 0              | 3386      | 568,471     | 0,40%               | 28,687    |

Tabela 3: Wyniki eksperymentu dla 1000 operacji

| n      | Best $\alpha$ | min. błąd | Worst $\alpha$ | max. błąd | średni błąd | Względny błąd (B&B) | Czas [ms] |
|--------|---------------|-----------|----------------|-----------|-------------|---------------------|-----------|
| 100(1) | 0,406         | 17        | 0              | 3358      | 538,944     | 1,92%               | 583,408   |
| 100(2) | 0,965         | 11        | 0              | 3312      | 635,197     | 1,15%               | 582,756   |
| 100(3) | 0,735         | 48        | 0              | 3697      | 802,562     | 13,95%              | 526,959   |
| 100(4) | 0,405         | 20        | 0              | 3584      | 447,8       | 14,23%              | 579,621   |
| 100(5) | 0,362         | 4         | 0              | 3386      | 546,937     | 0,27%               | 569,366   |

Wykonując ten test, można ulepszyć algorytm własny. Już przy wykonaniu tylko 50 operacji w 4 z 5 przypadków uzyskano lepszy wynik (porównując błąd względny). Minusem tego jest czas wykonania, który jest ok. 37,5 razy dłuższy.

Wykonano również testy dla 1000 operacji. Tylko w jednym przypadku udało się polepszyć wynik, którego błąd wyniósł 4 jednostki, a błąd względny tylko 0,27%. Jednak wielką wadą jest tutaj ogromny czas, dłuższy ok. 687,5 razy niż przy jednej iteracji.

## 4 Wnioski

- Czas wykonania jest najgorszy dla algorytmu zupełnego ze względu na jego złożoność obliczeniową, która jest postaci  $O(n!)$ . Inne algorytmy mają w większości złożoność logarytmiczną.
- Najgorszym algorytmem, którego błąd względny dla dużych rozmiarów instancji jest ogromny to sortowanie po  $r_j$ . Wynika to z nie uwzględniania pożądanego terminu zakończenia zadania.

- Lepszym algorytmem jest sortowanie po  $d_j$ , gdzie wyniki są już lepsze, jednak istnieją lepsze algorytmy niż heurystyczne, które wykorzystują np. kolejki priorytetowe.
- Jednym z najlepszych algorytmów jest algorytm Schrage, który zarówno patrzy na to, które zadanie jest aktualnie dostępne w kolejce priorytetowej, ale również wybiera te o najmniejszym czasie zakończenia.
- Dodając do algorytmu Schrage przerwania, można otrzymać jeszcze lepsze wyniki, lecz wiąże się to z przerywaniem i wybieraniem zadania w każdej chwili o najmniejszym czasie zakończenia.
- Algorytm Branch and Bound (Carliera) pozwala na znalezienia rozwiązania optymalnego, w miarę szybkim czasie. W dużych instancjach do jego wyniku porównuje się wyniki innych algorytmów.
- Algorytm własny z wybranym średnim parametrem współczynnika pozwolił na otrzymanie lepszych wyników niż algorytm Schrage. Po zastosowaniu pętli i wykonaniu wielu operacji, otrzymano lepszy współczynnik, który pozwolił algorytmowi na uzyskanie jeszcze dokładniejszych wyników kryterium.